

Problem Solving and Agile Software Development



agile adjective

ag·ile ('a-jəl -jī(-ə)l

Synonyms of *agile* >

- 1 : marked by ready ability to move with quick easy grace
| an *agile* dancer
- 2 : having a quick resourceful and adaptable character
| an *agile* mind

agilely ('a-jəl(-l)-lē -jī(-ə)(l)-lē adverb

Agile

- We will approach developing software with an agile mindset.
- This is to guide you on how to create and respond to change and how to deal with uncertainty.
- This will introduce you to how industry approaches building software.

What is Agile Software Development?

Agile is like Planning a Road Trip with Friends

- Goal/Destination: Agree on where you're heading.
- Flexible Plans: Decide details as you go.
- Team Collaboration: Everyone contributes ideas.
- Regular Check-ins: Stop often, adjust route if needed.
- Deliver in Pieces: Enjoy one stop before moving to the next.

Agile is about flexibility, feedback, and small steps — just like a trip with friends.

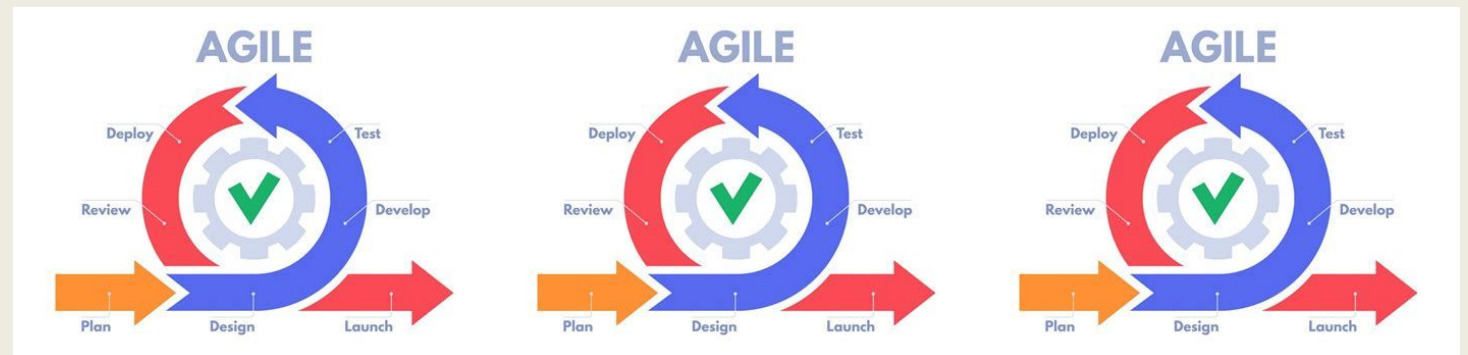
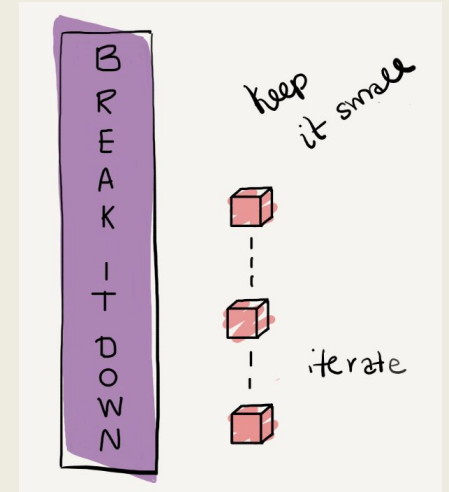


Problem Solving and Software Development

Approach problem solving using Agile Software Development concepts to help you design and implement a solution that is maintainable, testable and extendable.

You will

1. Design:
 - a. Analyze User Stories and Acceptance Criteria (requirements)
 - b. Break the problem into smaller tasks or into classes
 - c. Create test cases for tasks (methods)
 - d. Design Algorithms in Pseudocode for Tasks (Methods)
2. Develop and Unit Test
 - a. Unit test as you implement code
 - b. Create quality code
3. Complete final testing
4. Retrospective: Reflect and Improve
5. Iterate to add more features



Analyze → Plan → Design → write tests (pre/post, edge cases) → pseudocode → implement one method → run tests → refactor → next method → final test → retrospective

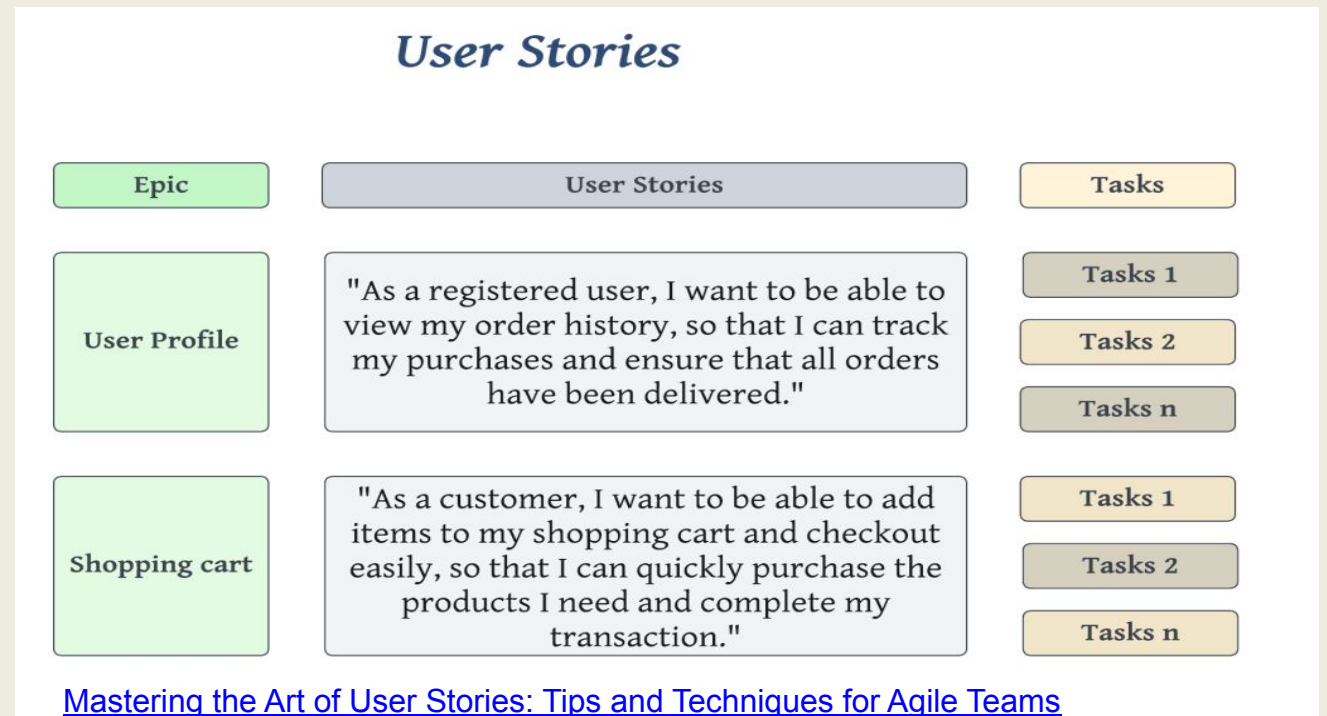
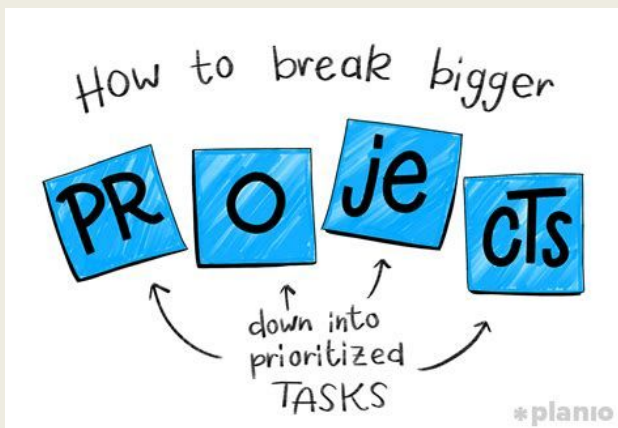
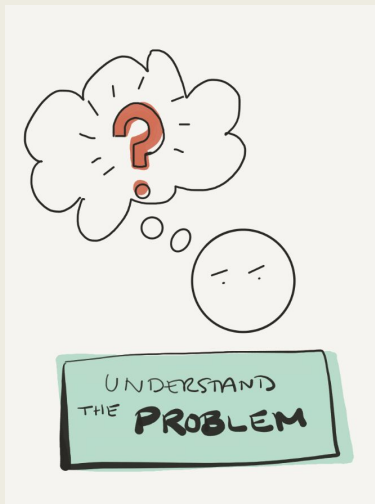
Design: Analyze Problem and Break into Smaller Tasks

Requirements: User Stories are what the user needs and acceptance criteria is how we know it works.

- A short sentence that describes a feature from the user's perspective.
- Format: As a [type of user], I want [some goal] so that [reason/value].

Modular programming is breaking down of a problem into smaller and simpler parts so you can design the algorithm for a method.

Smaller tasks (methods) provide a way to organize and simplify code to make it easier to maintain, change, test and debug.



Example User Story and Acceptance Criteria

User Story: As a professor, I want to enter category scores and get a final % and letter grade so that I don't have to calculate it manually.



Acceptance criteria (definition of “done”)

- Program prints category weights and grading scale.
- Each category grade is validated (0–105, re-prompt until valid).
- Final % is calculated and shown to two decimals.
- Letter grade is computed using the scale.
- User can repeat for another student (‘y’ = continue; otherwise exit).

Analyze the problem and break it into tasks.

Category:Percent

Class Participation: 10%
Guided Exploration: 20%
Module Quizzes: 25%
Project Iterations: 25%
Final Exam: 20%

Letter Grade Range

A: 90 to 100
B: 80 to < 90
C: 70 to < 80
D: 60 to < 70
F: < 60

Example: Break into Tasks

Task 1: Display summary of program including

- Grading categories and weights
- Grading Scale

Task 2: Enter & validate category grade

- When grade not 0 to 105 display error and repeat until valid grade entered

Task 3: Calculate final grade percentage

- Class Participation: 10%
- Guided Exploration: 20%
- Module Quizzes: 25%
- Project Iterations: 25%
- Final Exam: 20%

Task 4: Determine Final Letter Grade

- A: 90 to 100
- B: 80 to < 90
- C: 70 to < 80
- D: 60 to < 70
- F: < 60

Task 5: Repeat Calculating Grade for another student

- yes
- no

Unit Testing and Test Driven Development

Functionally



Unit Testing

A unit test checks one small piece of code (like a method) to make sure it works correctly.

Each test has:

- Precondition: Input (what you give the method)
- Postcondition: Expected Output (what you want back)

Test-Driven Development (TDD)

A way of writing software where writing tests come first before the actual code.

Steps

1. Write the minimum code to make the test pass
2. Refactor (clean up the code if needed to make it better)
3. Repeat for the next test

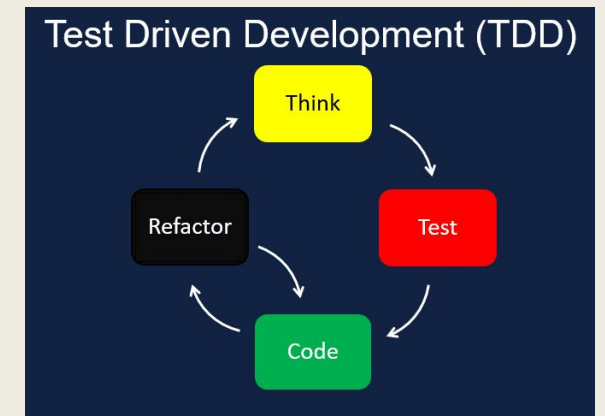
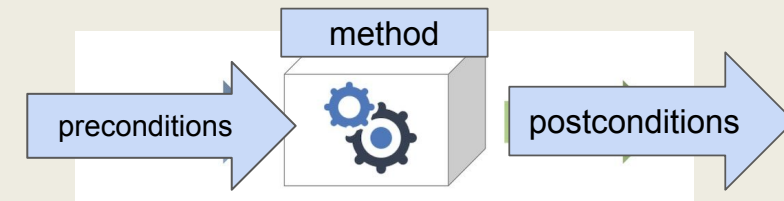
Summary

- Unit test = a small check that makes sure one method (or class) works correctly.
- TDD = instead of writing the method first, you write the unit test first. Then you write just enough code to make a test pass.

Why use TDD and Unit Tests?

- Helps you plan your methods (inputs/outputs).
- Catches mistakes early (before programs get big).
- Makes your code more reliable, reusable, and maintainable.

Resources: [What is Unit Testing? | Agile Alliance](#) and [Test Driven Development \(TDD\)](#)



Edge Cases Test Example Task 4

Edge cases are the borderline scores. If your code handles those correctly, it will usually handle all other inputs correctly too.

What are edge cases?

- Edge cases are inputs right at the boundary of a decision.
- They test whether your program handles the “just barely” situations correctly.

Example: A / B boundary to Determine Final Letter Grade :

- Input: 90.0 → Expect: A
- Input: 89.99 → Expect: B

What would be better test case?

Good edge test cases?

What are other good test cases?

Pre-Condition	Post Condition	Notes on Code Structure
finalGradePercent = 93.80	A	if finalGradePercent >= 90 letter grade = 'A'
finalGradePercent = 80	B	
finalGradePercent = 79.99	C	

Do you need
selection if/else, multiway if/else, nested if/else,
switch
or
iteration (looping) - while, do while, for

Letter Grade Range

A: 90 to 100

B: 80 to < 90

C: 70 to < 80

D: 60 to < 70

F: < 60

Software Design Principles

Single Responsibility Principle (SRP)

Each method or function should do one job only.

If a method tries to do many things, it becomes harder to test, fix, and reuse.

Example:

- Good: `calculateFinalPercentage()` only finds the average.
- Bad: `calculateFinalPercentage()` also prints results, asks for input, and writes to a file.

SRP makes code easier to understand, debug, and reuse.

Don't Repeat Yourself (DRY)

- Don't copy and paste the same logic in multiple places.
- If you need the same code in several places, put it in a method and call it.
- Example: Instead of checking for a valid grade in 5 spots, write `getValidGrade()` once and use it everywhere.
- DRY means less code, fewer bugs, easier updates.

Keep It Simple Silly (KISS)

- Code should be clear and simple, not clever or complicated.
- Computers don't care, but humans need to read and understand your code.
- Example: Instead of writing a giant method with lots of if statements, break it into small, readable methods.
- Simple code is easier to test, fix, and share with others.

When you write code, think SRP (one job per method), DRY (don't copy/paste), and KISS (keep it clear and simple). These principles make your programs easier to read, test, and improve.

Resources: [Software Design Principles DRY and KISS](#) and [Functions and the Single Responsibility Principle](#)

Design Algorithms in Pseudocode for Tasks (Methods)

After writing test cases, write an algorithm, a step by step procedure in pseudocode for method tasks including main method



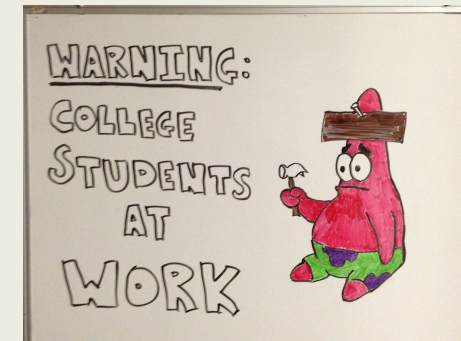
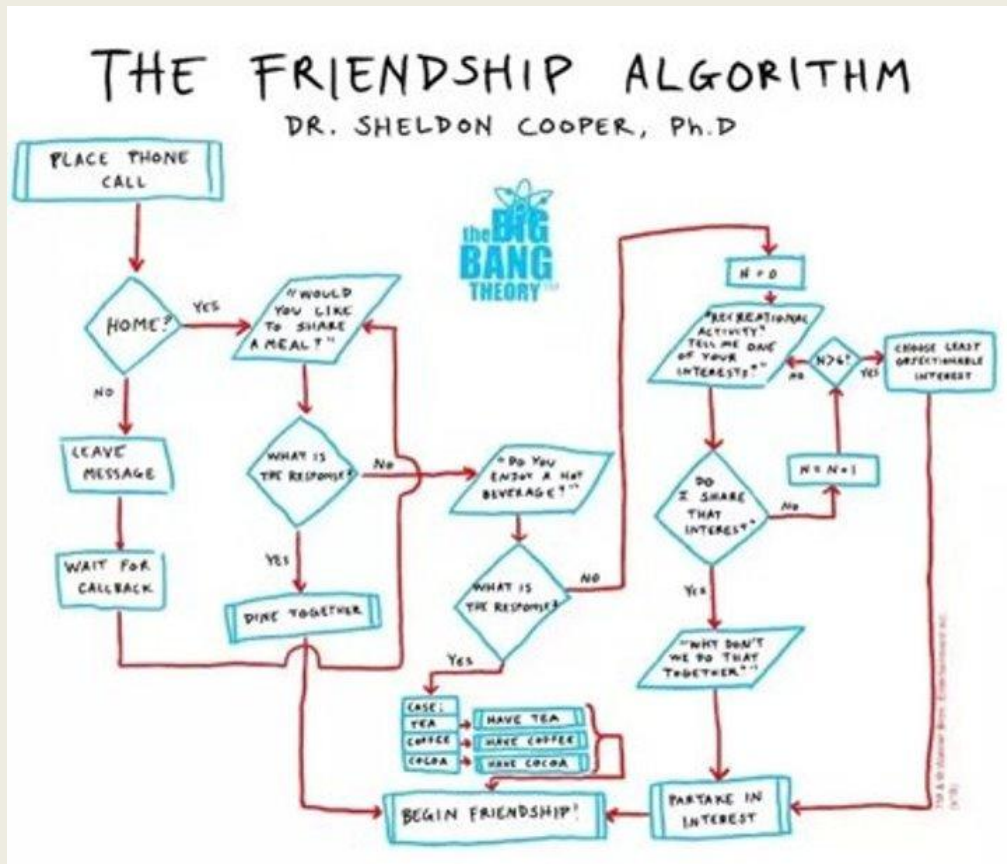
When Problem Solving



Before you start coding!

Using a writing tool and not typing to create pseudocode on paper, white board or tablet.

You will take a photo and insert into your design document



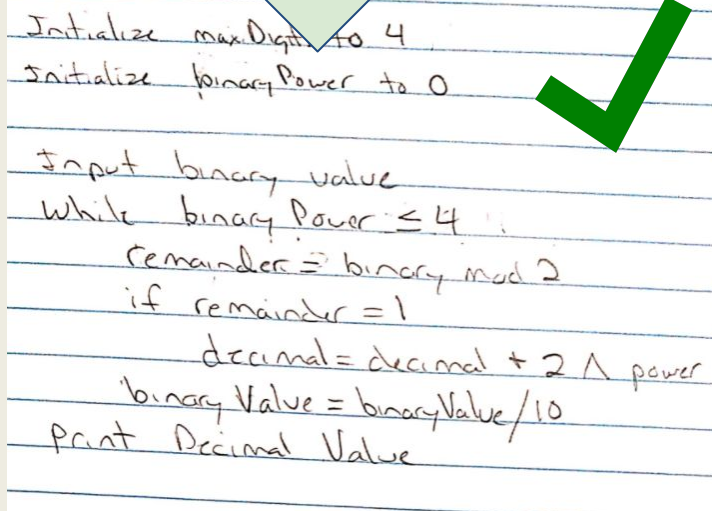
Design Algorithms using Pseudocode - NOT Actual Code

Pseudocode

- Acts as the bridge between your brain and computer code
- Pseudocode is an English version of what your program does.
- Pseudocode it is NOT pure code - it is a mixture of English and coding constructs
- Helps you think about the logic to solve the problem
- Can be used to solve problems in various language

Do

Use proper indentation
Include code structures but not
syntax

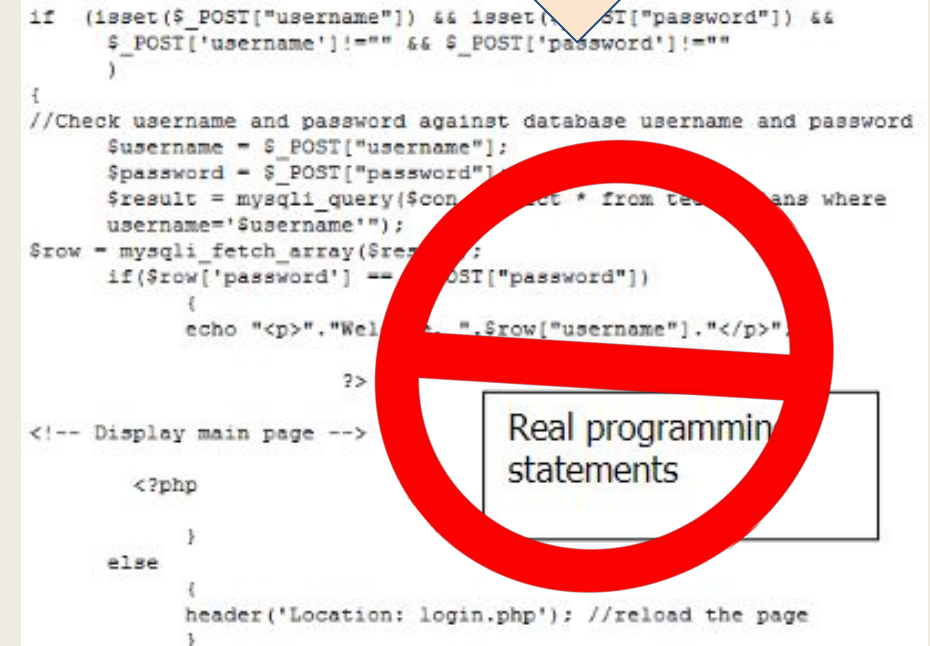


```
Initialize maxDigit to 4
Initialize binaryPower to 0

Input binary value
While binaryPower ≤ 4
    remainder = binary mod 2
    if remainder = 1
        decimal = decimal + 2 ^ power
    binaryValue = binaryValue / 10
Print Decimal Value
```

Don't

Use specific syntax
Write actual code



```
if (isset($_POST["username"]) && isset($_POST["password"]) &&
    $_POST['username'] != "" && $_POST['password'] != ""
)
{
    //Check username and password against database username and password
    $username = $_POST["username"];
    $password = $_POST["password"];
    $result = mysqli_query($con, "select * from test_users where
    username='$username'");
    $row = mysqli_fetch_array($result);
    if($row['password'] == $_POST["password"])
    {
        echo "<p>". "Welcome ". $row["username"]. "</p>";
    }
    ?>

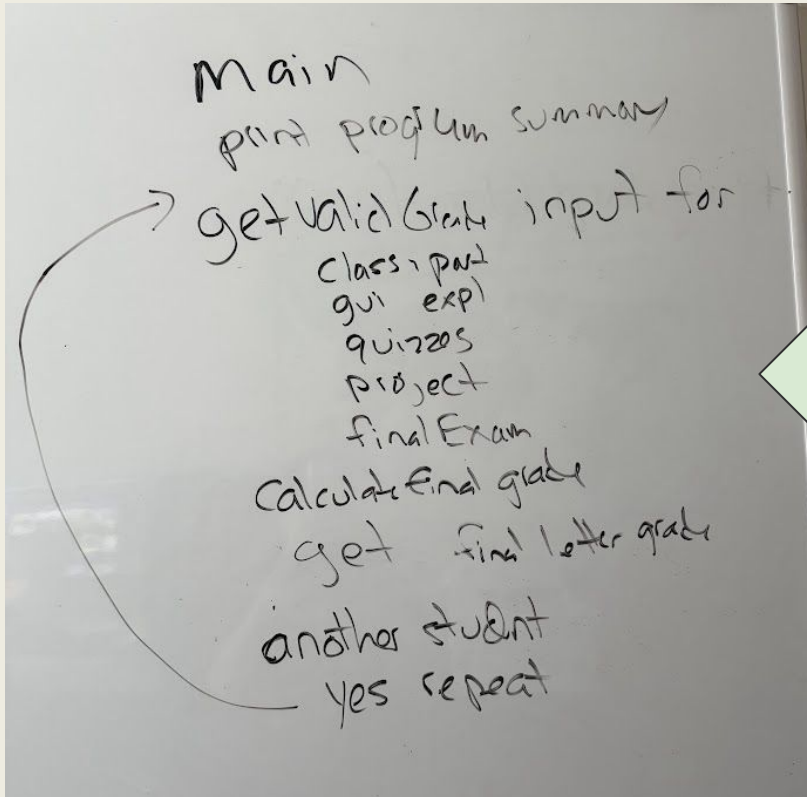
<!-- Display main page -->

<?php
}
else
{
    header('Location: login.php'); //reload the page
}
```

Real programming
statements

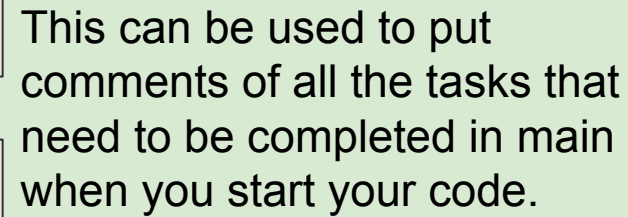
Design Main Method

Think about what methods to do the tasks need to be called from main.



A handwritten list of tasks for a main method on a piece of paper. The tasks are: 'main', 'print program summary', 'get valid grade input for', 'class', 'part', 'gui', 'expl', 'quizzes', 'project', 'final Exam', 'calculate final grade', 'get final letter grade', 'another student', and 'yes repeat'. A curved arrow starts from the bottom of the list and points back to the 'get valid grade input for' line, indicating a loop.

```
main
print program summary
get valid grade input for
class
part
gui
expl
quizzes
project
final Exam
calculate final grade
get final letter grade
another student
yes repeat
```



This can be used to put comments of all the tasks that need to be completed in main when you start your code.

Design Method Algorithm in Pseudocode

Look at Preconditions / Postconditions (edge cases first).

Method signature (return, name , parameters).

Think about

- **Name for the method (verb)**
- **Parameters (what the method needs to do the task)**
- **Return (what is returned after the task is done)**

Precondition	Postcondition
finalGradePercent = 90	A
finalGradePercent = 80	B
finalGradePercent = 79.99	C

Write Pseudocode (on paper, tablet or white board)

Method signature:

Multiway if algorithm for handling test cases

Assign char to letterGrade variable

Return variable holding correct char

```
Name: char determineFinalLetterGrade (double finalNumberGrade)
char letterGrade
if finalNumberGrade >= 90 -> letterGrade = A
else if finalNumberGrade >= 80 -> letterGrade = B
else if _____ >= 70 -> _____ = C
else if _____ >= 60 -> _____ = D
else (catch all) letterGrade = F
return letterGrade
```


Outline Main Method

Add comments to plan your main method.

Start adding constants and variables.

Add one method at a time and test.

```
public static void main(String[] args)
{
    // Constants for max and min grade range
    // Constants for grade weights
    // Initialize Variables to hold possible grade for each category
    // Task 1: Method to Display summary of program

    // Task 2: Enter & validate each category grade
    // Task 3: Calculate final grade percentage

    //Task 4: Get final letter grade

    //Task 5 while user answers yes repeat Tasks 1 to 4
```

Implement Pseudocode and Unit Test

Write a little code to pass one and test. Debug and and add more code.

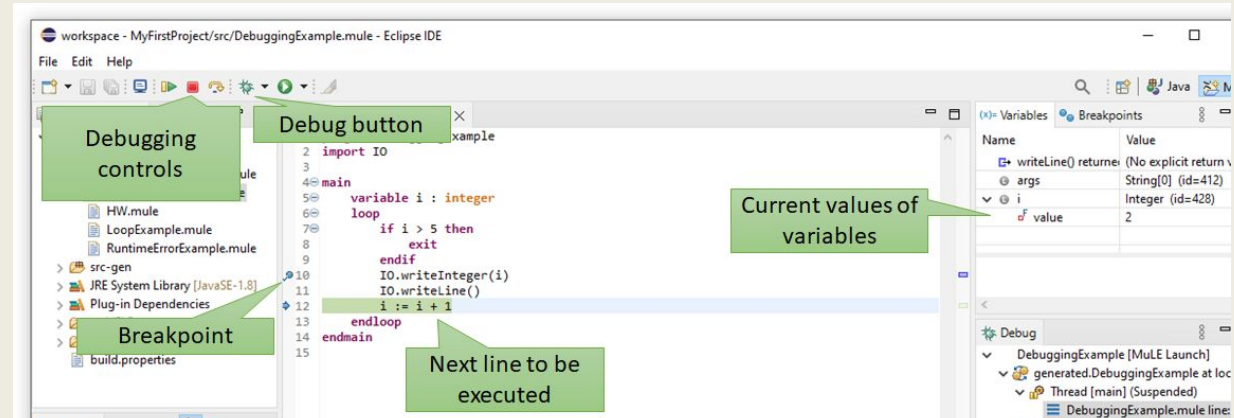
```
/**
 * Determines the letter grade for a student based on their final numeric grade.
 * Uses the standard grading scale: A: 90 to 100 B: 80 < 90 C: 70 < 80 D: 60 <
 * 70 F: below 60
 *
 * @param finalGrade the student's final grade as a double
 * @return the corresponding letter grade as a char ('A', 'B', 'C', 'D', or 'F')
 */
public static char getLetterGrade(double finalGrade)
{
    char letterGrade = ' ';
    if (finalGrade >= 90)
    {
        letterGrade = 'A';
    }
    return letterGrade;
}
```

← Add part of the algorithm to pass a test case



Precondition	Postcondition
finalGradePercent = 90	A
finalGradePercent = 80	B
finalGradePercent = 79.99	C

Call method from main
Pass hardcoded values to unit
test the method.



```
public static void main(String[] args) {
```

```
//Unit test get letter grade
```

```
//Hardcode unit test precondition created in your design
```

```
//Use debugger to see if it stores the correct letter grade
```

```
char letterGrade = getLetterGrade(90);
```


Quality Coding Guide (Following Oracle and Google Standards)



Read [Code Conventions for the Java Programming Language: 1. Introduction](#)

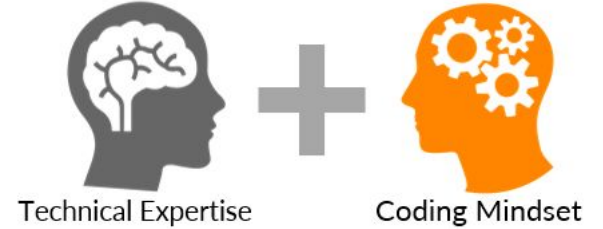
We will follow the following when implementing code

- [Naming Conventions - Java](#)

- Class name - FancyCars
- Methods - driveOnlyOnSunday()
- Variables - numberCylinders
- constant - MAX_STUDENTS



Good programming is a combination of



- [Braces are used with if, else, for, do and while statements, even when the body is empty or contains only a single statement.](#)

```
if (finalGrade >= 90)
{
    letterGrade = 'A';
} else if (finalGrade >= 80)
```

- [One constant or variable declaration per line since it encourages commenting](#) and [Put constant and variable declarations at the beginning of method blocks and initialize.](#)

```
// Constants for grade weights
final double CLASS_PART_PERCENT = .15;
final double GUIDE_EXPL_PERCENT = .2;
// Initialize Variables to hold possible grade for each category
double guideExpl = 0;
double classPart = 0;
```

- Do not use **ternary operator ?** for this class. While the ternary operator is useful for simple conditions, it can make code harder to read if overused or used with complex logic. In such cases, traditional if-else statements may be clearer.

Avoid Hardcoding

Hardcoding can be used when testing but your final program should have hard coded values also called “magic numbers” .

They make code hard to read, hard to update, and easy to break.

```
while (grade < 0 || grade > 105) {  
    System.out.println("Invalid grade. Grade min is 0 and max is 105");  
    grade = input.nextDouble();  
}
```

If 105 changes you must find every 105 and change it.
It is easy to miss one and cause bugs.

Instead, use constants, variables, or parameters so your code is clear, maintainable, and easily modified.

```
public static void main(String[] args)  
{  
    final int MIN_GRADE = 0;  
    final int MAX_GRADE = 105;
```

```
// Get all grades for students while Using Valid grade method  
// to make sure grade entered is between max and min
```

```
System.out.print("Enter Class Participation grade: ");  
classParticipation = enterValidGrade(MIN_GRADE, MAX_GRADE);  
System.out.print("Enter Guided Exploration grade: ");  
guidedExploration = enterValidGrade(MIN_GRADE, MAX_GRADE);
```

Now if max and min change
you only update one place
and it will work everywhere

```
public static double enterValidGrade(int currentMin, int currentMax)  
{  
    Scanner input = new Scanner(System.in);  
    double grade = 0;  
    while (grade < currentMin || grade > currentMax)  
    {  
        System.out.println("Invalid grade. " + "Grade min is "  
            + currentMin + "and max is " + currentMax);  
        System.out.print("Re-enter grade: ");  
        grade = input.nextDouble();  
    }  
    input.close();  
    return grade;  
}
```

Document Code

Documenting with informative comments

```
1 /*  
2  * Name: [YOUR NAME]  
3  * Class: CS1050 (M/W or T/TH)  
4  * Description: Guided Exploration 01  
5  *  
6  * Ask the professor for category grades for each student.  
7  * If a category grade is not a value 0 to 100 then give an error message and the program will end  
8  * If valid values are entered for the category grades then  
9  * Calculate final grade percentage  
10 * Displays final percentage for student grade  
11 * Display final letter grade  
12 * Display a table of grading scale and weighted categories  
13 */  
14  
15 import java.util.Scanner; //used to read input from the keyboard  
16  
17 public class HardingDebGE02  
18 {
```

1: Initial comment at top of source file should include name, class and description of what program does.

2: Comments above blocks of code to explain complex code

```
//calculate final grade percentage by adding up each  
//each student category score times the percentage weight  
finalGrade = (learningActivity * LEARN_ACT_WEIGHT) + (homework * homeworkWeight) +  
              (quiz * quizWeight) + (midterm * midtermWeight) + (finalExam * finalExamWeight)
```

Notice how the built in methods contain comments

Character.isDigit

isDigit(char ch) : boolean - Character

Determines if the specified character is a digit.

Parameters:

ch the character to be tested.

Returns:

true if the character is a digit; false otherwise.

3. Comments above method you create should contain summary of what method does and describe the parameters and return value

```
/**  
 * Determines the letter grade for a student based on their final numeric grade.  
 * Uses the standard grading scale:  
 * A: 90 to 100  
 * B: 80 < 90  
 * C: 70 < 80  
 * D: 60 < 70  
 * F: below 60  
 *  
 * @param finalGrade the student's final grade as a double  
 * @return the corresponding letter grade as a char ('A', 'B', 'C', 'D', or 'F')  
 */  
public static char getLetterGrade(double finalGrade)
```

[Generate JavaDoc Comments](#) for methods and update with details

Implement Structured Programming

For the purpose of this class the goal is quality software over best-performing so you are writing structured programming

- Do not use infinite loops break/continue/return/exit in a loop or if/else condition to cause an immediate exit.
- Breaks can be used in case statements.
- Only use one return at the end of a method.

```
public static char getLetterGrade(double finalGrade){  
    char letterGrade = ' ';  
  
    if (finalGrade >= 90){  
        letterGrade = 'A';  
    } else if (finalGrade >= 80){  
        letterGrade = 'B';  
    } else if (finalGrade >= 70){  
        letterGrade = 'C';  
    } else if (finalGrade >= 60){  
        letterGrade = 'D';  
    } else{  
        letterGrade = 'F';  
    }  
    return letterGrade;  
}
```



✓ Good Approach to exiting loops

Use a proper loop condition

Let the loop end naturally

Use if-else instead

Ensure loop condition is correct

```
public static char getGrade(double score) {  
    if (score >= 90.0)  
        return 'A';  
    else if (score >= 80.0)  
        return 'B';  
    else if (score >= 70.0)  
        return 'C';  
    else if (score >= 60.0)  
        return 'D';  
    else  
        return 'F';  
}
```



✗ Bad Approach exiting loops

Using **break** unnecessarily

Using **System.exit(0)**

Using **continue** unnecessarily

Creating an **infinite loop**

```
while (true)  
{  
    printf("Wash your hands.\n");  
}
```



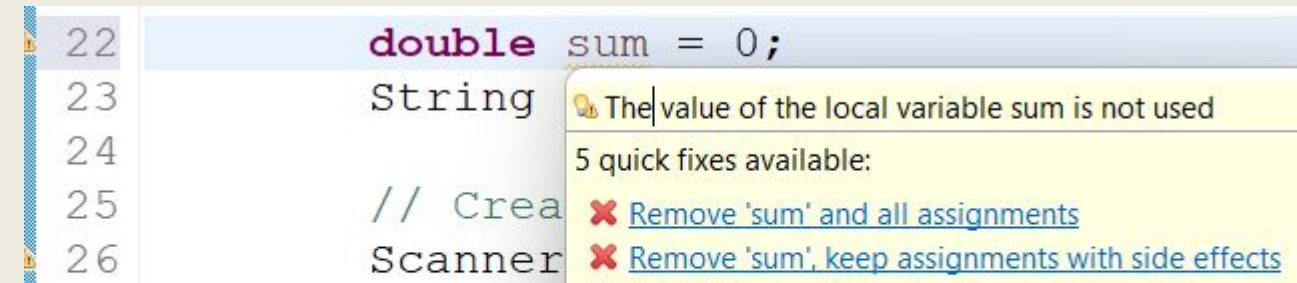
Eliminate Warnings

When you finish implementing your code your program should not contain warnings — they can indicate possible security risks and make code harder to maintain. Fixing them builds professional habits.



```
24 // create a scanner object to get  
25 Scanner input = new Scanner(System.in);  
26  
27 //create  
28 System.out.println("Enter a number:");
```

Resource leak: 'input' is never closed
4 quick fixes available:



```
22 double sum = 0;  
23 String  
24  
25 // Crea  
26 Scanner
```

The value of the local variable sum is not used
5 quick fixes available:
✗ [Remove 'sum' and all assignments](#)
✗ [Remove 'sum', keep assignments with side effects](#)

Integration Testing

1. Unit Testing (Already completed when implementing methods.

- Tests one method or small piece of code.
- Does each method work correctly on its own for the varying preconditions, postconditions, and edge cases.
- Example: test `getValidGrade()` or `getLetterGrade()`.

After Unit testing test how the methods are integrated to make sure the entire system work for the user stories and acceptance criteria.

2. Integration Testing

- Tests how multiple methods work together.
- Do the methods connect correctly? Does data flow correctly between them?
- Common issues: wrong parameters passed, type mismatches, or logic errors between methods.

Example for the Grade Calculator:

- Does it work correctly in calculating one students grade?
- Does it work correctly when enter yes or know to calculate another students grade?



Does each unit
(method) work

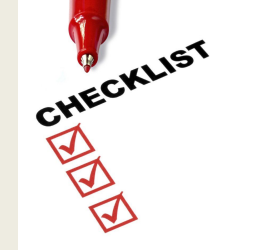


Do the units
(methods work
together)

Summary

Following this industry process helps

1. Build Maintainable and Testable Code
 - b. Modular (Broken into smaller pieces)
 - c. Reusable following Single Responsibility Principle (SRP) and Don't Repeat Yourself (DRY)
 - d. Flexible (Not Hard coded)
 - e. Read ability: Follows coding standars and documented properly
2. Measure that the solution correctly satisfies all user stories and acceptance criteria
 - b. Unit Tests
 - c. Integration Tests



Reflect and Celebrate

What is a Retrospective? Reflect and discuss after the iteration

Celebrate successes
Lessons learned
Plan for improvements

Have you acknowledged all your
hard work and accomplishments



Plan Your Time

Look at your tasks and plan dates you will work on each task

Spreadsheet for [Project Planning](#) or [What you can do using project planning on GitHub - DEV Community](#)

SUNDAY	MONDAY	TUESDAY	WEDNESDAY	THURSDAY	FRIDAY	SATURDAY
					1 List tasks to code and unit test	2
3 List Tasks to code and unit test	4	5 Finish coding and testing	6	7 Finish coding and testing	8	9 Review assignment details and if complete submit
10	11 DUE!	12	13	14	15	16